Zero-Trust Credential Relay

Secure Credential Management for Organizations

The Problem

Organizations face a critical security gap when sharing login credentials across teams:

- **Shared passwords** live in spreadsheets, Slack DMs, or sticky notes
- **No audit trail** — nobody knows who used which credential, when, or from where
- **No control** — once a password is shared, it can't be unshared
- **Credential sprawl** — employees copy passwords into browsers, password managers, and notes
- **Zero accountability** — if a breach happens, there's no way to trace it

Our Solution: Zero-Trust Credential Relay

A system where **credentials are never seen or stored by employees** — they flow directly from an admin-controlled vault into the login form, with full audit logging.

Core Principle

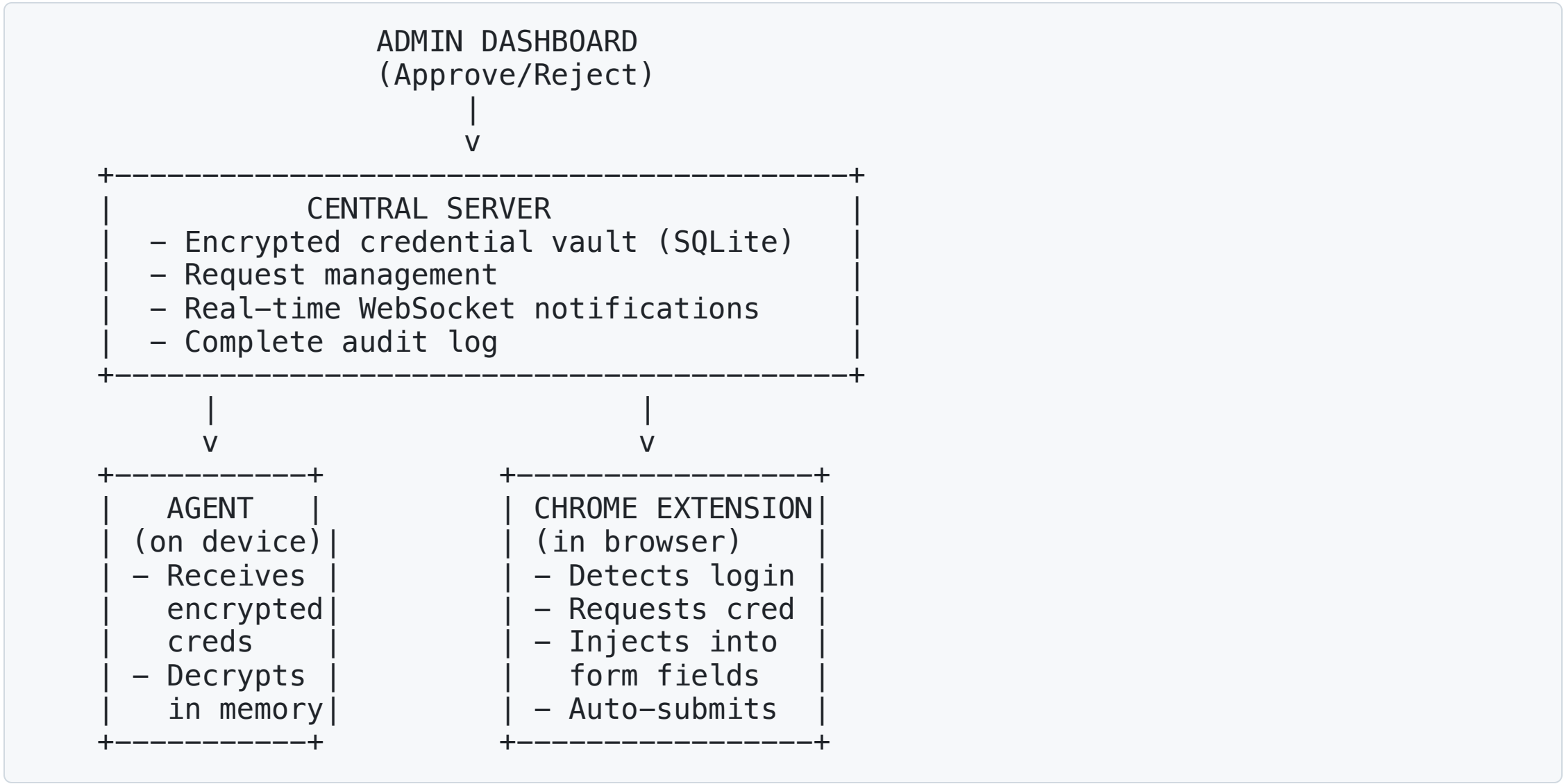
> "The employee uses the credential. They never possess it."

How It Works (Simple Version)

```
1. Employee visits a login page (e.g., LeetCode, AWS Console)
2. Extension detects the login form automatically
3. Employee requests access to a specific credential
4. Admin sees the request on the dashboard and approves (or rejects)
5. Credential is injected directly into the form — employee never sees the password
6. Form auto-submits — employee is logged in
7. Everything is logged: who, what, when, where, which device
```

Think of it like a hotel key card — you get access to the room, but you never see the physical lock mechanism. And the hotel knows exactly when you entered.

System Architecture



Security Features

Encryption

- Passwords stored using **AES-256-GCM** encryption (military-grade)
- Decryption happens **only in memory** on the employee's device
- Credentials are **never written to disk** in plaintext

Zero-Knowledge Employee

- Employee **never sees** the actual password
- Password field is **locked** after injection (copy/paste disabled, show-password blocked)
- Credential is **wiped from memory** after use or after 60-second TTL

Audit Trail

Live Demo Flow

What the Admin Sees (Dashboard)

1. Manage credentials (add/remove login accounts)
2. See registered devices with MAC addresses
3. Approve or reject credential requests in real-time
4. View complete audit log of all activity

What the Employee Experiences

1. Navigate to login page
2. Click extension → Select credential → Click "Request"
3. Wait a few seconds for admin approval
4. Form fills and submits automatically
5. They're logged in — without ever seeing the password

Risk Assessment

Current Risks (Prototype Stage)

Risk	Severity	Mitigation Path
Server runs on HTTP (localhost)	Medium	Deploy with HTTPS/TLS in production
No authentication on API endpoints	High	Add JWT auth + role-based access control
SQLite single-file database	Low	Migrate to PostgreSQL for production
Admin password in .env file	Medium	Use secret management (AWS KMS, Vault)
Single admin role	Medium	Add role hierarchy (super-admin, admin, viewer)

Real-World Implementation Plan

Phase 1: Internal Pilot (1-2 months)

- Deploy server on internal infrastructure (HTTPS, behind VPN)
- Add JWT authentication and role-based access
- Onboard one team (5-10 employees) for testing
- Replace PostgreSQL for production database
- **Cost: Minimal — internal developer time only**

Phase 2: Organization-Wide Rollout (2-3 months)

- SSO integration (Google Workspace / Azure AD) for admin login
- Mobile approval app for admins (push notifications)
- Bulk credential import from existing password managers
- Employee onboarding automation (device enrollment)

Competitive Advantage Over Existing Solutions

Feature	Our System	Password Managers	Shared Spreadsheets
Employee sees password	No	Yes	Yes
Per-use approval	Yes	No	No
Audit trail	Full	Partial	None
Auto-injection	Yes	Yes	No
Device tracking	Yes	No	No
Credential revocation	Instant	Delayed	Impossible
Cost	Low	$4-8/user/month	Free but dangerous

Tech Stack Summary

- **Backend**: Node.js + Express + TypeScript
- **Database**: SQLite (prototype) → PostgreSQL (production)
- **Encryption**: AES-256-GCM with scrypt key derivation
- **Real-time**: WebSocket for instant notifications
- **Browser**: Chrome Extension (Manifest V3)
- **Logging**: Pino (structured JSON logs)
- **Validation**: Zod schema validation

Summary

Zero-Trust Credential Relay solves a real, everyday security problem:

1. **Employees get access** — without seeing passwords
2. **Admins keep control** — approve every use in real-time
3. **Organization gets visibility** — complete audit trail of who used what, when, and from where
4. **Security is enforced** — not just recommended

> "Trust is good. Zero trust is better."

Built by ITGeeks — Credential Relay v1.0